Clinical Trial Matching System

Project Implementation & Architecture Guide

1. Executive Summary

The **Autonomous Medical & Clinical Trial Patient Matching System** is an advanced AI orchestration pipeline designed to eliminate the highly manual, error-prone process of matching patients to active clinical trials. In the real world, researchers spend weeks reading unstructured Electronic Health Records (EHR) to see if a patient meets the strict inclusion and exclusion criteria of a trial.

This system automates the workflow by extracting medical context from raw documents, securely analyzing it using local Large Language Models to maintain HIPAA compliance, and mapping the patient's profile against complex trial databases. Finally, it uses Machine Learning to predict the likelihood of the patient dropping out of the trial, ensuring only the most viable candidates are recommended.

> **Impact for Resumes:** This project demonstrates mastery of Agentic Workflows, Hybrid Vector Search, privacy-preserving local LLM usage, and traditional ML classification—highly sought-after skills in top-tier tech companies.

2. High-Level Architecture

The system operates in a multi-stage pipeline, orchestrating different agents to handle specific parts of the matching process:

1. **Ingestion & Parsing Phase:** Unstructured patient files (doctor's notes, PDFs) and Clinical Trial protocol documents are uploaded and parsed into structured text.
2. **Vectorization & Retrieval (Agentic RAG):** The parsed text is chunked and stored in a vector database. When a match is initiated, the system retrieves relevant medical conditions, biomarkers, and medications from both the patient profile and the trial criteria.
3. **Multi-Agent Orchestration:**
    - *Inclusion Agent:* Checks if the patient has the required conditions (e.g., "Stage II Melanoma").
    - *Exclusion Agent:* Checks for disqualifying factors (e.g., "Currently on immunosuppressants").
4. **ML Risk Scoring:** A trained classifier evaluates demographic data (age, distance from hospital) to output an attrition risk score.
5. **Output:** A formatted justification report explaining exactly why a patient is a match, citing the specific lines in the EHR.

3. Prerequisites & Free-Tier Tech Stack

To ensure this prototype can be built and showcased completely free of charge, the architecture utilizes generous free tiers and robust open-source tools.

Component	Tool/Service	Why it's Free & Ideal
Vector Database	Neon Database (PGVector)	Provides a generous free tier for serverless Postgres with native `pgvector` support, perfect for scalable vector storage.
LLM / AI Engine	Ollama (Llama 3 / Mistral)	Runs entirely locally on your machine. 100% free, zero API costs, and ensures patient data (even mock data) never leaves your device.
AI Orchestration	LangGraph & LangChain	Open-source Python frameworks to build the state machines and coordinate the Inclusion/ Exclusion agents.
ML Classification	LightGBM or Scikit-Learn	Open-source Python libraries to build the fast, tabular data classifier for the patient dropout prediction.
UI & Hosting	Vercel (Next.js)	The hobby tier allows seamless, zero-cost deployment of the frontend dashboard and API routing.

4. Step-by-Step Implementation Guide

Phase 1: Environment & Tool Setup

- **Install Ollama:** Download and install Ollama on your local machine. Pull a lightweight model to start by running `ollama run llama3:8b` or `mistral`.
- **Database Setup:** Set up a free Neon database project. Enable the vector extension by running `CREATE EXTENSION vector;` in the SQL editor. Connect your Python environment using standard Postgres libraries like `psycopg2`.
- **Project Repository:** Initialize a Python environment, installing `langchain`, `langgraph`, `sentence-transformers` (for local embeddings), and `lightgbm`.

Phase 2: The Agentic RAG Engine (Data Processing)

- **Mock Data Creation:** Generate 5-10 synthetic patient EHR PDFs and 3 mock Clinical Trial protocol text files.
- **Document Chunking:** Write a Python script using LangChain's `RecursiveCharacterTextSplitter` to break the PDFs into smaller paragraphs.
- **Embedding & Storage:** Use a free local embedding model (like HuggingFace's `all-MiniLM-L6-v2`) to convert chunks into vectors and push them into your Neon PGVector tables.

Phase 3: Building the LangGraph Orchestrator

- **Define the State:** Create a Python dictionary representing the state (Patient ID, Trial ID, Match Status, Justification).
- **Build the Agents:**
  - Write a prompt for the *Inclusion Agent*: "Based on the retrieved patient vector context, does the patient meet these trial criteria?"
  - Write a prompt for the *Exclusion Agent*: "Review the patient context for any of the following disqualifying conditions."
- **Graph Workflow:** Wire them together using LangGraph so the data flows from Retrieval → Inclusion Check → Exclusion Check.

Phase 4: The ML Attrition Model

- **Dataset:** Create a simple CSV with mock historical data (Age, Distance to Clinic, Number of Prior Treatments, Status: Completed/Dropped).
- **Training:** Train a simple LightGBM or Random Forest model using Scikit-Learn to predict the 'Status' column.
- **Integration:** Wrap the trained model (saved as a `.pkl` file) into a Python function that the Orchestrator calls as the final step.

Phase 5: User Interface Integration

- **Frontend:** Build a clean, interactive UI. A web interface deployed on Vercel allows you to upload a patient file, select a clinical trial, and trigger the backend pipeline.
- **Showcase:** The UI should display the agent's thought process step-by-step (e.g., "Extracting Data...", "Checking Inclusion...", "Calculating ML Risk...") to visually demonstrate the orchestration for recruiters.

5. Final Impact Summary

By completing this project, you will have built a full-stack, AI-native application that mimics enterprise-level data processing. You are demonstrating the ability to handle unstructured data via Vector Databases, utilize Agentic Workflows for complex reasoning, and integrate predictive Machine Learning—all while keeping infrastructure costs at zero for the prototype phase.